Raspberry Pi Arcade Joystick

A working arcade controller built from a door stop, four nails and a block of wood, wired straight into a Raspberry Pi.

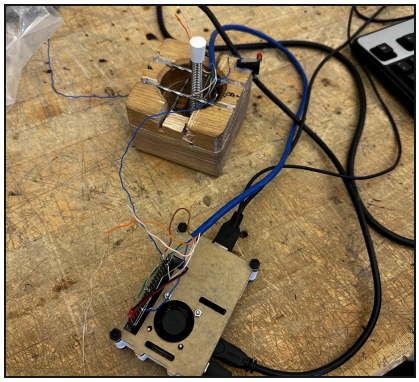


FIG. 4.1 — Joystick and cased Pi, Jan 2026

CONTEXT	ROLE	TIMELINE	METHODS
Independent project	Design, machining, wiring, configuration	2 weeks set · < 1 used	Drill mill · GPIO · input mapping · debouncing

4 + 1	<1 wk	0	2
DIRECTIONS PLUS FIRE BUTTON	TO FINISH A TWO-WEEK BUILD	OFF-THE-SHELF JOYSTICK PARTS	GAMES USED TO TEST IT: TETRIS AND ALIENS

01 BRIEF

Take an idea from concept to a working prototype using household parts instead of a kit. A store-bought joystick hides every interesting problem inside a plastic shell; building one from scratch meant solving the mechanical, electrical and software sides together, and seeing how a decision in one shows up in the others.

02 CONSTRAINTS

PARTS	**What was on hand. No joystick kit, no microswitches**
TIME	**Two weeks allotted**
INTERFACE	**Must bind like a normal gamepad in an arcade front end**
FEEL	**Self-centring, equal throw in all four directions**

03 HOW IT WORKS

The spring from a door stop is the stick: flexible enough to push, stiff enough to snap back to centre. Four nails stand around it, one per direction. Push the stick far enough and it touches a nail, which closes a circuit to one of the Pi's GPIO inputs. Electrically, the spring is the common contact and each nail is an independent input: touching a nail pulls that pin low, and releasing it lets the pull-up return it high. In effect, four normally-open switches made from hardware-store parts.

On the software side, an existing GPIO-to-gamepad mapping tool presents the pins to the arcade front end as a standard controller. My part was configuration: mapping each header pin to a direction (Fig. 4.4) and tuning the debounce timing until the stick felt right.

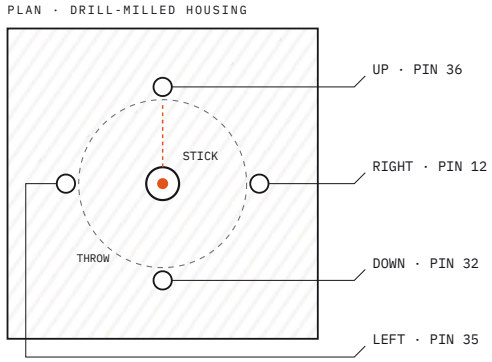


FIG. 4.2 — Contact layout. Each nail sits where the stick meets it at the end of its throw, equally in every direction. Pin numbers are physical header pins, from the mapping in Fig. 4.4.

04 PROCESS

01 Design around what was on hand
Door-stop spring for a self-centring stick; nails as contacts. No kit, no microswitches.

02 Lay out and machine the housing
Measured, marked and clamped the wooden block, then drilled it on a drill mill so every nail landed at the same distance from the stick, following machine-shop safety practice.

03 Wire it to the Pi
Each direction and the fire button to its own GPIO input.

04 Configure and tune the input layer
Map pins to directions, bind the controls, tune debounce in real games.

Contact bounce	FAILURE MODE 4.A
OBSERVED	One push of the stick registered as a burst of presses. In game, movement was jittery and laggy.
CAUSE	Metal-on-metal contacts don't close cleanly. The nail and spring chatter for a few milliseconds before settling, and the Pi reads fast enough to see every chatter as a separate press.
RESOLVED	Tuned the debounce timing in the mapping tool's configuration, so a state change only counts once the contact has held steady, and cleaned up the signal routing between the contacts and the header.
VERIFIED	Each push registered exactly once, with no visible lag, in play testing with Tetris and Aliens.

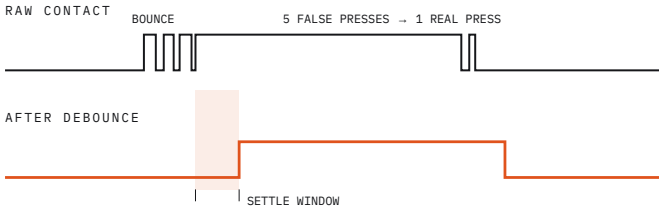


FIG. 4.5 — Illustrative timing, not measured. A change is only accepted after the signal holds for the settle window.

05 DECISIONS AND TRADE-OFFS

DECISION	INSTEAD OF	WHY
Nail contacts	Microswitches	On hand and free, as the brief required. The cost (bounce, wear) could be paid in software.
Door-stop spring	Custom spring and pivot	Self-centring for free, with a usable throw.
Direct GPIO	USB encoder board	No extra hardware, and it forces you to deal with the raw signal yourself.
Drill-milled wood	Hand drill or 3D print	Accurate, repeatable hole placement with the tools available.

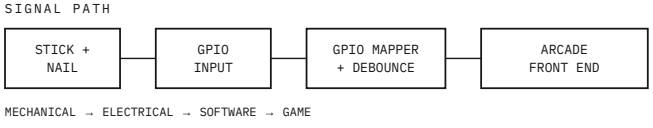
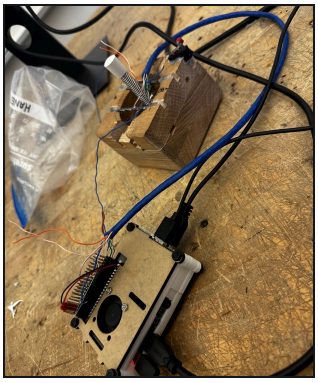
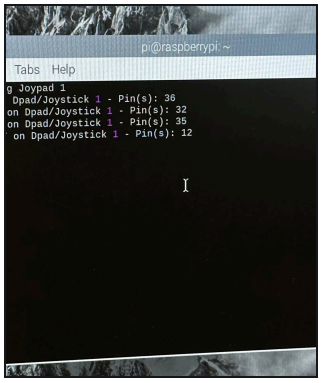


FIG. 4.3 — One push crosses three domains before it moves a block.



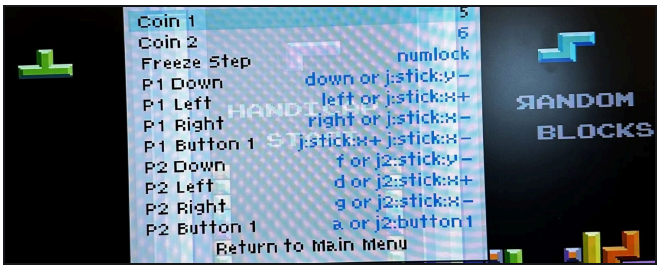
4.6 Stick in the housing



4.4 Direction → pin mapping

06 TWO WEEKS, DONE IN UNDER ONE

The project was scoped for two weeks and finished in under one. The time came from not working in series: the input mapping was tested on the Pi with the bare contacts while the housing was still being laid out, so by the time the stick was mounted, the software side was already known to work. The remaining time went into the part that only shows up once everything is assembled: how the controller actually feels to play.



4.7 Controls bound in the arcade front end

07 OUTCOME

A fully working arcade controller, used to play Tetris and Aliens, finished in under a week on a two-week timeline. Every physical part, from the stick's spring to the contacts and housing, was designed and made by hand, and the software layer was configured and tuned around it.

> *Small changes in the software had a butterfly effect on how the whole system responded. Hardware and software can't be debugged in isolation.*

08 REV. B

- Swap the nails for microswitches, so the contacts stop wearing and bouncing.
- Model the housing in SolidWorks and cut it on a CNC or 3D printer for a repeatable fit.
- Add a second action button for two-button games.
- Replace the configured tool with my own Python input layer, for direct control over debouncing and timing.